

Taller — Window Functions en PostgreSQL

Universidad de Pamplona

Facultad de Ingenierías y Arquitectura

Ingeniería de Sistemas

Bases de Datos II

2026-2

Estudiante:

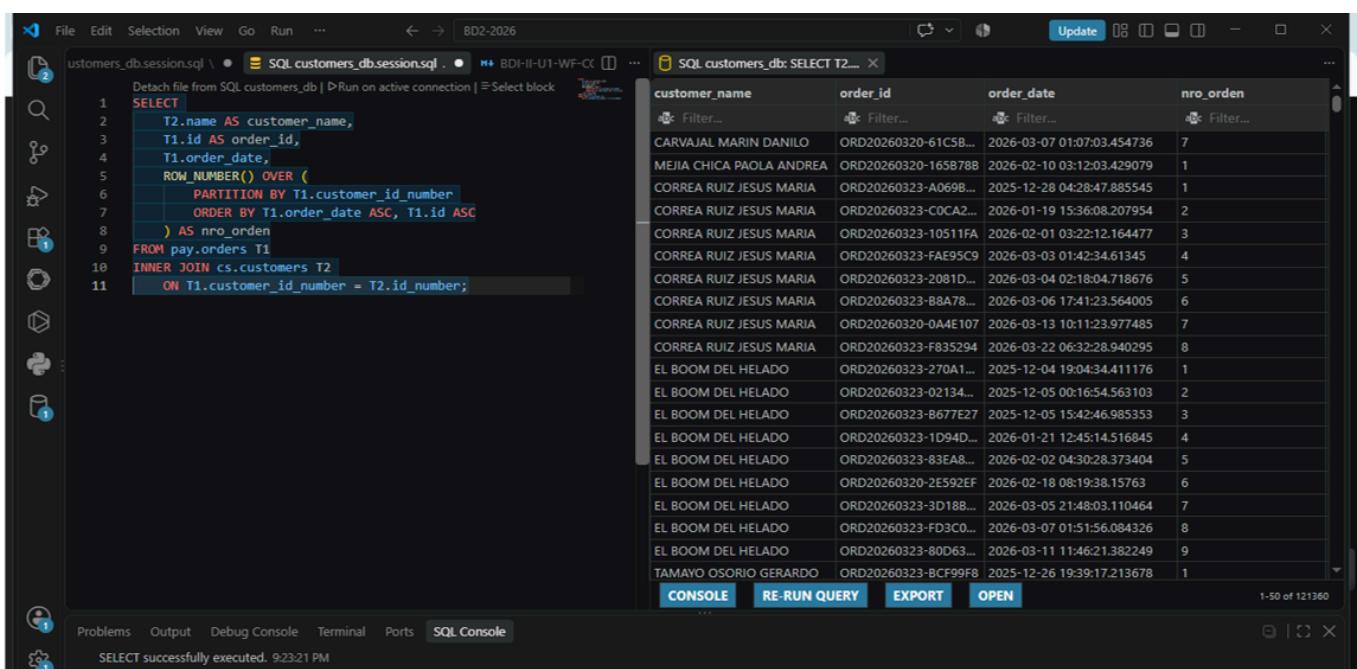
Nombre: Arley Ruben Martinez Mendoza

Fecha: 29 de septiembre de 2026 [Repositorio de GitHub - BD2](#)

Ejercicio 1. ROW_NUMBER — Historial de compras por cliente

a. Consulta por cada orden de pay.orders

```
SELECT
  T2.name AS customer_name,
  T1.id AS order_id,
  T1.order_date,
  ROW_NUMBER() OVER (
    PARTITION BY T1.customer_id_number
    ORDER BY T1.order_date ASC, T1.id ASC
  ) AS nro_orden
FROM pay.orders T1
INNER JOIN cs.customers T2
  ON T1.customer_id_number = T2.id_number;
```



The screenshot shows a PostgreSQL IDE interface. On the left, a SQL query is displayed in a dark-themed editor. The query is a SELECT statement that joins the 'pay.orders' table (aliased as T1) with the 'cs.customers' table (aliased as T2) using an INNER JOIN on the 'customer_id_number' column. The query uses the ROW_NUMBER() window function to assign a sequential number (nro_orden) to each order within each customer's partition, ordered by order_date and then order_id. On the right, the results of the query are shown in a table view. The table has four columns: customer_name, order_id, order_date, and nro_orden. The results list 20 orders from various customers, including CARVAJAL MARIN DANILO, MEJIA CHICA PAOLA ANDREA, CORREA RUIZ JESUS MARIA, and EL BOOM DEL HELADO. The bottom status bar indicates that the query was successfully executed at 9:23:21 PM.

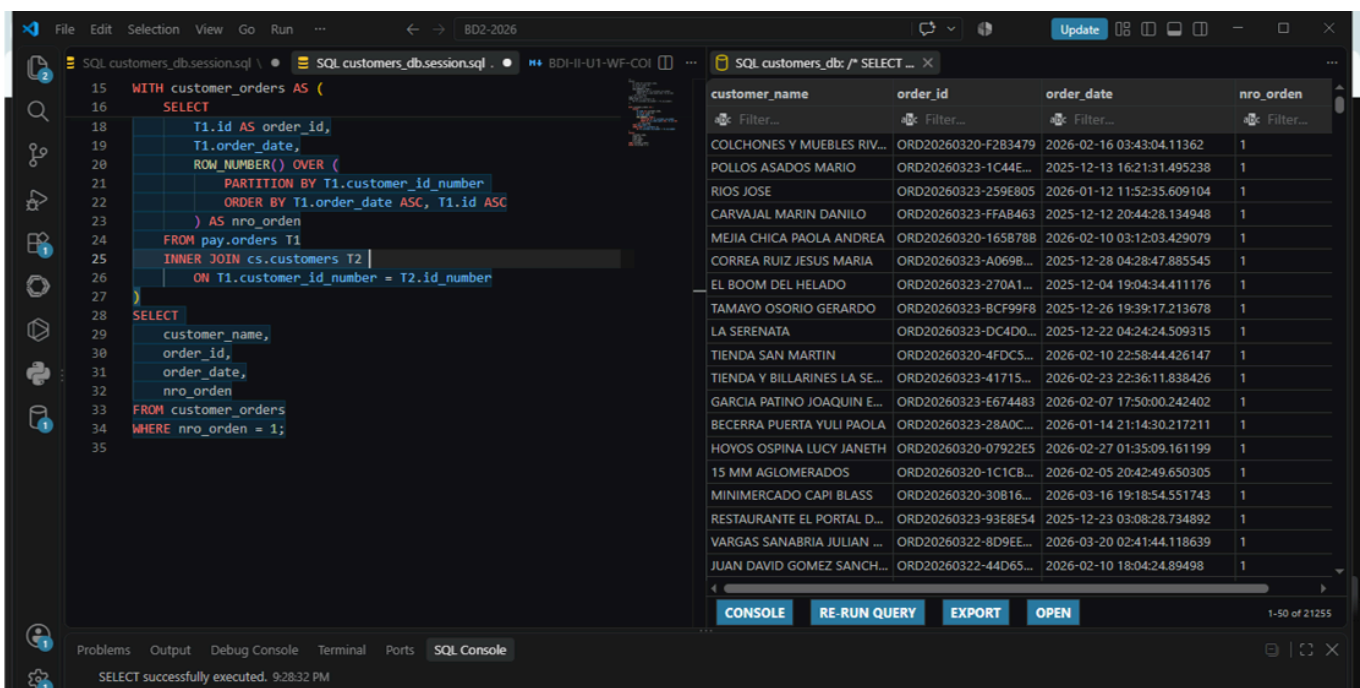
customer_name	order_id	order_date	nro_orden
CARVAJAL MARIN DANILO	ORD20260320-61C58...	2026-03-07 01:07:03.454736	7
MEJIA CHICA PAOLA ANDREA	ORD20260320-1658788	2026-02-10 03:12:03.429079	1
CORREA RUIZ JESUS MARIA	ORD20260323-A0698...	2025-12-28 04:28:47.885545	1
CORREA RUIZ JESUS MARIA	ORD20260323-C0CA2...	2026-01-19 15:36:08.207954	2
CORREA RUIZ JESUS MARIA	ORD20260323-10511FA	2026-02-01 03:22:12.164477	3
CORREA RUIZ JESUS MARIA	ORD20260323-FAE95C9	2026-03-03 01:42:34.61345	4
CORREA RUIZ JESUS MARIA	ORD20260323-2081D...	2026-03-04 02:18:04.718676	5
CORREA RUIZ JESUS MARIA	ORD20260323-B8A78...	2026-03-06 17:41:23.564005	6
CORREA RUIZ JESUS MARIA	ORD20260320-0A4E107	2026-03-13 10:11:23.977485	7
CORREA RUIZ JESUS MARIA	ORD20260323-F835294	2026-03-22 06:32:28.940295	8
EL BOOM DEL HELADO	ORD20260323-270A1...	2025-12-04 19:04:34.411176	1
EL BOOM DEL HELADO	ORD20260323-02134...	2025-12-05 00:16:54.563103	2
EL BOOM DEL HELADO	ORD20260323-B677E27	2025-12-05 15:42:46.985353	3
EL BOOM DEL HELADO	ORD20260323-1D94D...	2026-01-21 12:45:14.516845	4
EL BOOM DEL HELADO	ORD20260323-83EA8...	2026-02-02 04:30:28.373404	5
EL BOOM DEL HELADO	ORD20260320-2E592EF	2026-02-18 08:19:38.15763	6
EL BOOM DEL HELADO	ORD20260323-3D188...	2026-03-05 21:48:03.110464	7
EL BOOM DEL HELADO	ORD20260323-FD3C0...	2026-03-07 01:51:56.084326	8
EL BOOM DEL HELADO	ORD20260323-80D63...	2026-03-11 11:46:21.382249	9
TAMAYO OSORIO GERARDO	ORD20260323-BCF99F8	2025-12-26 19:39:17.213678	1

b. CTE y filtro

```

WITH customer_orders AS (
  SELECT
    T2.name AS customer_name,
    T1.id AS order_id,
    T1.order_date,
    ROW_NUMBER() OVER (
      PARTITION BY T1.customer_id_number
      ORDER BY T1.order_date ASC, T1.id ASC
    ) AS nro_orden
  FROM pay.orders T1
  INNER JOIN cs.customers T2
    ON T1.customer_id_number = T2.id_number
)
SELECT
  customer_name,
  order_id,
  order_date,
  nro_orden
FROM customer_orders
WHERE nro_orden = 1;

```



customer_name	order_id	order_date	nro_orden
COLCHONES Y MUEBLES RIV...	ORD20260320-F2B3479	2026-02-16 03:43:04.11362	1
POLLOS ASADOS MARIO	ORD20260323-1C4E...	2025-12-13 16:21:31.495238	1
RIOS JOSE	ORD20260323-259E805	2026-01-12 11:52:35.609104	1
CARVAJAL MARIN DANILO	ORD20260323-FFAB463	2025-12-12 20:44:28.134948	1
MEJIA CHICA PAOLA ANDREA	ORD20260320-165B78B	2026-02-10 03:12:03.429079	1
CORREA RUIZ JESUS MARIA	ORD20260323-A069B...	2025-12-28 04:28:47.885545	1
EL BOOM DEL HELADO	ORD20260323-270A1...	2025-12-04 19:04:34.411176	1
TAMAYO OSORIO GERARDO	ORD20260323-BCF99F8	2025-12-26 19:39:17.213678	1
LA SERENATA	ORD20260323-DC4D0...	2025-12-22 04:24:24.509315	1
TIENDA SAN MARTIN	ORD20260320-4FDC5...	2026-02-10 22:58:44.426147	1
TIENDA Y BILLARINES LA SE...	ORD20260323-41715...	2026-02-23 22:36:11.838426	1
GARCIA PATINO JOAQUIN E...	ORD20260323-E674483	2026-02-07 17:50:00.242402	1
BECERRA PUERTA YULI PAOLA	ORD20260323-28A0C...	2026-01-14 21:14:30.217211	1
HOYOS OSPINA LUCY JANETH	ORD20260320-07922E5	2026-02-27 01:35:09.161199	1
15 MM AGLOMERADOS	ORD20260320-1C1CB...	2026-02-05 20:42:49.650305	1
MINIMERCADO CAPI BLASS	ORD20260320-30B16...	2026-03-16 19:18:54.551743	1
RESTAURANTE EL PORTAL D...	ORD20260323-93E8E54	2025-12-23 03:08:28.734892	1
VARGAS SANABRIA JULIAN ...	ORD20260322-8D9EE...	2026-03-20 02:41:44.118639	1
JUAN DAVID GOMEZ SANCH...	ORD20260322-44D65...	2026-02-10 18:04:24.89498	1

c. Análisis

Las funciones de ventana como ROW_NUMBER() se procesan al final del flujo de SQL, después del WHERE, GROUP BY y HAVING. Por eso, PostgreSQL no permite evaluarlas dentro del WHERE y genera un error de sintaxis (ERROR: window functions are not allowed in WHERE).

Ejercicio 2. RANK y DENSE_RANK — Empresas de envío por volumen.

a. consulta sobre ship.shipment_orders y ship.ship_company

```

SELECT
    T2.name AS company_name,
    COUNT(T1.id) AS total_shipments,
    RANK() OVER (ORDER BY COUNT(T1.id) DESC) AS rank_normal,
    DENSE_RANK() OVER (ORDER BY COUNT(T1.id) DESC) AS rank_dense
FROM ship.shipment_orders T1
INNER JOIN ship.ship_company T2
    ON T1.ship_company_id = T2.id
GROUP BY T2.id, T2.name
ORDER BY total_shipments DESC;

```

The screenshot shows a SQL IDE with the query from the previous block executed. The results are displayed in a table with the following data:

company_name	total_shipments	rank_normal	rank_dense
Interrapidísimo S.A.	12256	1	1
Coordinadora Mercantil S.A.	12243	2	2
Lozano Hermanos Transportes	12202	3	3
Envía Carga S.A.S.	12171	4	4
DHL Express Colombia Ltda.	12160	5	5
Deprisa - Avianca Cargo	12147	6	6
TCC Envíos S.A.S.	12103	7	7
FedEx Colombia S.A.S.	12075	8	8
Servientrega S.A.	12025	9	9
UPS Colombia S.A.S.	11977	10	10

b. Identificación y análisis de empates

Resultado actual: No hubo empates en los resultados (van de 12,256 a 11,977 envíos). Por eso, RANK() y DENSE_RANK() generaron exactamente la misma secuencia del 1 al 10.

Si hubiera un empate en el puesto 3:

RANK() dejaría un hueco y saltaría al puesto 5 para la siguiente empresa (cuenta competidores superados).

DENSE_RANK() mantendría la secuencia y asignaría el puesto 4.

c. Modificación de consulta

```

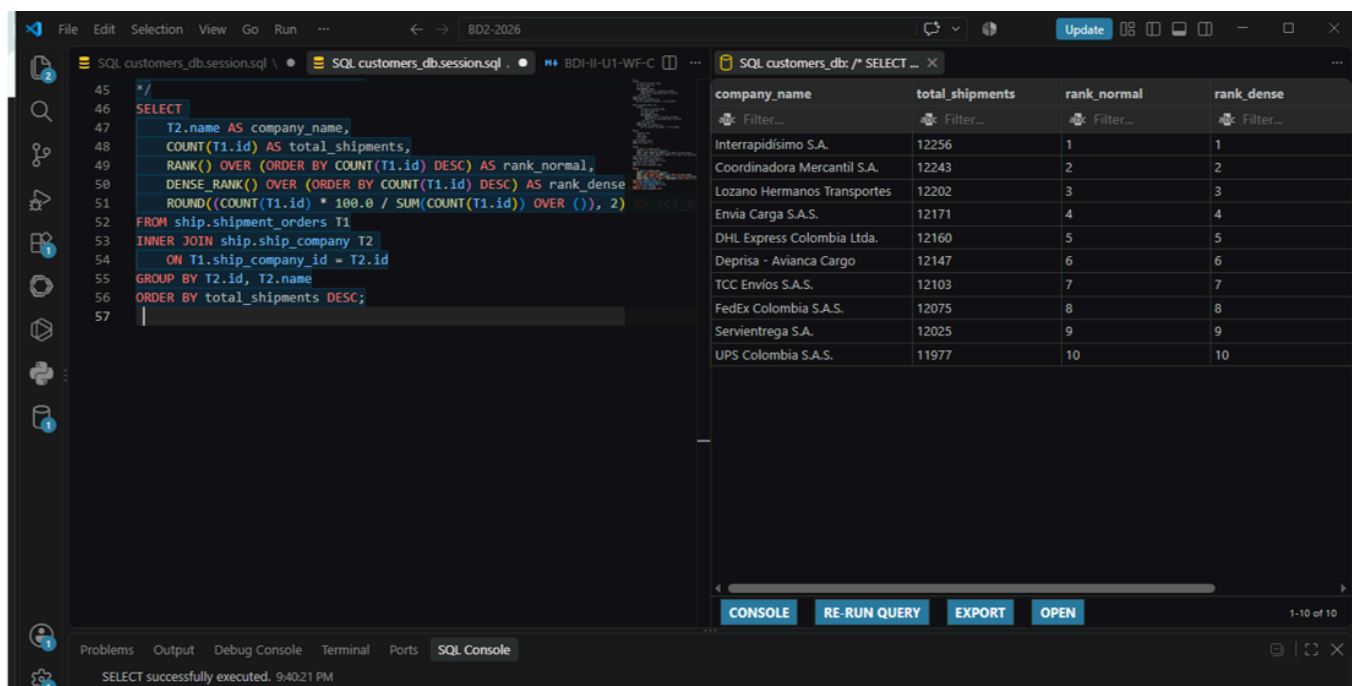
SELECT
    T2.name AS company_name,
    COUNT(T1.id) AS total_shipments,
    RANK() OVER (ORDER BY COUNT(T1.id) DESC) AS rank_normal,
    DENSE_RANK() OVER (ORDER BY COUNT(T1.id) DESC) AS rank_dense,
    ROUND((COUNT(T1.id) * 100.0 / SUM(COUNT(T1.id)) OVER ()), 2) AS pct_of_total
FROM ship.shipment_orders T1

```

```

INNER JOIN ship.ship_company T2
  ON T1.ship_company_id = T2.id
GROUP BY T2.id, T2.name
ORDER BY total_shipments DESC;

```



The screenshot shows an IDE with an SQL query editor on the left and a results pane on the right. The query is as follows:

```

45 SELECT
46   T2.name AS company_name,
47   COUNT(T1.id) AS total_shipments,
48   RANK() OVER (ORDER BY COUNT(T1.id) DESC) AS rank_normal,
49   DENSE_RANK() OVER (ORDER BY COUNT(T1.id) DESC) AS rank_dense,
50   ROUND((COUNT(T1.id) * 100.0 / SUM(COUNT(T1.id) OVER ()), 2)
51 FROM ship.shipment_orders T1
52 INNER JOIN ship.ship_company T2
53   ON T1.ship_company_id = T2.id
54 GROUP BY T2.id, T2.name
55 ORDER BY total_shipments DESC;
56
57

```

The results pane displays a table with the following data:

company_name	total_shipments	rank_normal	rank_dense
Interrapidísimo S.A.	12256	1	1
Coordinadora Mercantil S.A.	12243	2	2
Lozano Hermanos Transportes	12202	3	3
Envia Carga S.A.S.	12171	4	4
DHL Express Colombia Ltda.	12160	5	5
Deprisa - Avianca Cargo	12147	6	6
TCC Envíos S.A.S.	12103	7	7
FedEx Colombia S.A.S.	12075	8	8
Servientrega S.A.	12025	9	9
UPS Colombia S.A.S.	11977	10	10

At the bottom of the IDE, a status bar indicates: "SELECT successfully executed, 9:40:21 PM".

d. Criterio de uso

Usar RANK(): Cuando el puesto representa una posición absoluta y penaliza a los siguientes por los empates previos.

Ejemplo: Premios o incentivos a los 3 primeros. Si hay un empate en el 2.º lugar, el siguiente es el 4.º mejor en rendimiento real, por lo que no debe recibir premio de 3.

Usar DENSE_RANK() (Sin saltos): Cuando se clasifican elementos por niveles o categorías continuas.

Ejemplo: Categorización por Niveles de Servicio (Nivel 1, Nivel 2, Nivel 3). Garantiza que no se salten niveles en la escala operativa.

Ejercicio 3. SUM OVER sin partición - Participación en el ingreso total

a. consulta sobre pay.order_items y ctg.products

```

SELECT
  T2.name AS product_name,
  SUM(T2.cop_price * T1.quantity) AS total_revenue_cop,
  SUM(SUM(T2.cop_price * T1.quantity)) OVER () AS grand_total,
  ROUND(
    (SUM(T2.cop_price * T1.quantity) * 100.0 / SUM(SUM(T2.cop_price *
T1.quantity)) OVER ()),
    2
  ) AS pct_of_total,

```

```

ROUND(
    (SUM(SUM(T2.cop_price * T1.quantity)) OVER (ORDER BY SUM(T2.cop_price *
T1.quantity) DESC) * 100.0
    / SUM(SUM(T2.cop_price * T1.quantity)) OVER ()),
    2
) AS cumulative_pct
FROM pay.order_items T1
INNER JOIN ctg.products T2
    ON T1.product_id = T2.id
GROUP BY T2.id, T2.name
ORDER BY total_revenue_cop DESC;

```

product_name	total_revenue_cop	grand_total	pct_of_total
MacBook Air M2	131587252500.00	1621186265250.00	8.12
iPhone 14 128GB	87238815000.00	1621186265250.00	5.38
Laptop Lenovo IdeaPad 15	84548247675.00	1621186265250.00	5.22
Drone DJI Mini 3	82942571250.00	1621186265250.00	5.12
Televisor Samsung 55" 4K QLED	74941537500.00	1621186265250.00	4.62
Laptop HP Pavilion 14	71910903750.00	1621186265250.00	4.44
Cámara Sony ZV-E10 con Lente Kit	64918132500.00	1621186265250.00	4.00
Licencia Adobe Creative Cloud 1 año	64834919400.00	1621186265250.00	4.00
Laptop Asus VivoBook 16	63158736037.50	1621186265250.00	3.90
Televisor LG 50" NanoCell 4K	61562443162.50	1621186265250.00	3.80
Lentes VR PlayStation VR2	59182886250.00	1621186265250.00	3.65
Meta Quest 3 128GB	54715155675.00	1621186265250.00	3.38
Mini PC Intel NUC i5	45665550000.00	1621186265250.00	2.82
Silla Gamer Secretlab TITAN	43490002500.00	1621186265250.00	2.68
Monitor LG 27" 4K UHD	42909457500.00	1621186265250.00	2.65
Gafas AR Xreal Air 2	41897028750.00	1621186265250.00	2.58
Samsung Galaxy A54 128GB	38730768375.00	1621186265250.00	2.39
Aspiradora Robot Roomba i3	32675660775.00	1621186265250.00	2.02
Auriculares Sony WH-1000XM5	31384429087.50	1621186265250.00	1.94

b. Verificación columna grand_total

La columna grand_total muestra exactamente el mismo valor en todas las filas de la consulta. Esto sucede porque la cláusula OVER () se encuentra totalmente vacía, sin PARTITION BY ni ORDER BY.

c. Modificación de consulta

Al agregar ORDER BY SUM(T2.cop_price * T1.quantity) DESC dentro de la ventana de la columna cumulative_pct, PostgreSQL calcula una suma acumulada progresiva fila por fila en orden descendente de ventas, dividiendo cada acumulado entre el grand_total global para obtener la participación porcentual acumulada.

Ejercicio 4. SUM OVER con ORDER BY - Ingreso acumulado mes a mes

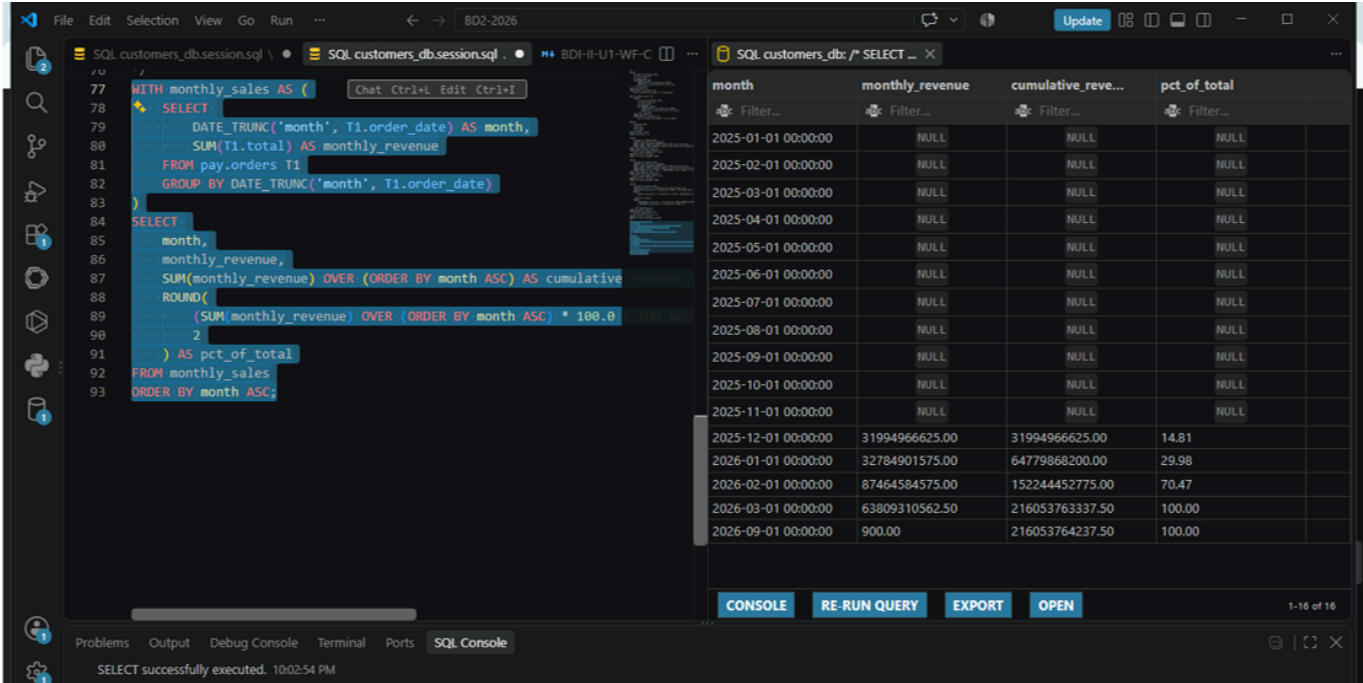
a. y b. Consulta de ventas acumuladas por mes y porcentaje global

```

WITH monthly_sales AS (
    SELECT
        DATE_TRUNC('month', T1.order_date) AS month,

```

```
        SUM(T1.total) AS monthly_revenue
    FROM pay.orders T1
    GROUP BY DATE_TRUNC('month', T1.order_date)
)
SELECT
    month,
    monthly_revenue,
    SUM(monthly_revenue) OVER (ORDER BY month ASC) AS cumulative_revenue,
    ROUND(
        (SUM(monthly_revenue) OVER (ORDER BY month ASC) * 100.0 /
    SUM(monthly_revenue) OVER ()),
        2
    ) AS pct_of_total
FROM monthly_sales
ORDER BY month ASC;
```



c. Análisis escrito y tabla de flujo acumulado

Cuando se incluye ORDER BY dentro del OVER() de una función de agregación como SUM(), PostgreSQL aplica de forma implícita el marco de ventana BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. Esto significa que el acumulado de la fila n incluye su propio valor más la suma de todas las filas previas, convirtiéndose directamente en la base del valor acumulado para la fila n+1.

Mes (month)	Ventas del Mes (monthly_revenue)	Acumulado (cumulative_revenue)	Flujo de Acumulación
Fila 1 (2025-12)	V_1	V_1	Base inicial
Fila 2 (2026-01)	V_2	$V_1 + V_2$	$\downarrow$ (Base anterior $V_1 + V_2$)

Mes (month)	Ventas del Mes (monthly_revenue)	Acumulado (cumulative_revenue)	Flujo de Acumulación
Fila 3 (2026-02)	V_3	$V_1 + V_2 + V_3$	↓ (Base anterior ($V_1 + V_2$) + V_3)
Fila 4 (2026-03)	V_4	$V_1 + V_2 + V_3 + V_4$	↓ (Base anterior ($V_1 + V_2 + V_3$) + V_4)

d. Experimento con PARTITION BY payment_method_id

```
WITH monthly_sales AS (
  SELECT
    DATE_TRUNC('month', T1.order_date) AS month,
    T1.payment_method_id,
    SUM(T1.total) AS monthly_revenue
  FROM pay.orders T1
  GROUP BY DATE_TRUNC('month', T1.order_date), T1.payment_method_id
)
SELECT
  month,
  payment_method_id,
  monthly_revenue,
  SUM(monthly_revenue) OVER (
    PARTITION BY payment_method_id
    ORDER BY month ASC
  ) AS cumulative_revenue
FROM monthly_sales
ORDER BY payment_method_id, month ASC;
```

Al incorporar PARTITION BY payment_method_id, el conjunto de datos se fragmenta en grupos independientes por cada método de pago. El cálculo del acumulado mensual opera de forma aislada dentro de cada grupo y reinicia su contador en 0 cada vez que la consulta pasa a evaluar un nuevo método de pago

The screenshot shows a SQL IDE with a query editor on the left and a results pane on the right. The query in the editor is as follows:

```

95 WITH monthly_sales AS (
96     SELECT
97         DATE_TRUNC('month', T1.order_date) AS month,
98         T1.payment_method_id,
99         SUM(T1.total) AS monthly_revenue
100     FROM pay.orders T1
101     GROUP BY DATE_TRUNC('month', T1.order_date), T1.payment_method_id
102 )
103 SELECT
104     month,
105     payment_method_id,
106     monthly_revenue,
107     SUM(monthly_revenue) OVER (
108         PARTITION BY payment_method_id
109         ORDER BY month ASC
110     ) AS cumulative_revenue
111 FROM monthly_sales
112 ORDER BY payment_method_id, month ASC;
113

```

The results pane displays a table with the following data:

month	payment_metho...	monthly_revenue	cumulative_reve...
2025-01-01 00:00:00	1	NULL	NULL
2025-02-01 00:00:00	1	NULL	NULL
2025-03-01 00:00:00	1	NULL	NULL
2025-04-01 00:00:00	1	NULL	NULL
2025-05-01 00:00:00	1	NULL	NULL
2025-06-01 00:00:00	1	NULL	NULL
2025-07-01 00:00:00	1	NULL	NULL
2025-08-01 00:00:00	1	NULL	NULL
2025-09-01 00:00:00	1	NULL	NULL
2025-10-01 00:00:00	1	NULL	NULL
2025-11-01 00:00:00	1	NULL	NULL
2025-12-01 00:00:00	1	3370564462.50	3370564462.50
2026-01-01 00:00:00	1	3031106100.00	6401670562.50
2026-02-01 00:00:00	1	9148835362.50	15550505925.00
2026-03-01 00:00:00	1	6370031437.50	21920537362.50
2026-09-01 00:00:00	1	900.00	21920538262.50

The IDE interface includes a top menu bar, a toolbar with icons for file operations, and a bottom status bar showing "SELECT successfully executed. 10:12:19 PM".

Ejercicio 5. LAG - Variación mensual de ventas

a. Consulta con LAG, diferencia y variación porcentual

```

WITH monthly_sales AS (
    SELECT
        DATE_TRUNC('month', T1.order_date) AS month,
        SUM(T1.total) AS sales
    FROM pay.orders T1
    GROUP BY DATE_TRUNC('month', T1.order_date)
)
SELECT
    month,
    sales AS current_month_sales,
    COALESCE(LAG(sales) OVER (ORDER BY month ASC), 0) AS prev_month_sales,
    sales - COALESCE(LAG(sales) OVER (ORDER BY month ASC), 0) AS difference,
    ROUND(
        CASE
            WHEN LAG(sales) OVER (ORDER BY month ASC) IS NULL THEN 0.00
            ELSE ((sales - LAG(sales) OVER (ORDER BY month ASC)) * 100.0 /
LAG(sales) OVER (ORDER BY month ASC))
        END,
        2
    ) AS pct_change
FROM monthly_sales
ORDER BY month ASC;

```

The screenshot shows a SQL IDE with a query editor on the left and a results grid on the right. The query is a complex SQL statement using CTEs and window functions to calculate monthly sales and percentage changes. The results grid displays data for various months, with the most recent month (2026-09-01) showing a significant decrease in sales.

month	current_month_sales	prev_month_sales	difference	pct_change
2025-01-01 00:00:00	NULL	0	NULL	0.00
2025-02-01 00:00:00	NULL	0	NULL	0.00
2025-03-01 00:00:00	NULL	0	NULL	0.00
2025-04-01 00:00:00	NULL	0	NULL	0.00
2025-05-01 00:00:00	NULL	0	NULL	0.00
2025-06-01 00:00:00	NULL	0	NULL	0.00
2025-07-01 00:00:00	NULL	0	NULL	0.00
2025-08-01 00:00:00	NULL	0	NULL	0.00
2025-09-01 00:00:00	NULL	0	NULL	0.00
2025-10-01 00:00:00	NULL	0	NULL	0.00
2025-11-01 00:00:00	NULL	0	NULL	0.00
2025-12-01 00:00:00	31994966625.00	0	31994966625.00	0.00
2026-01-01 00:00:00	32784901575.00	31994966625.00	789934950.00	2.47
2026-02-01 00:00:00	87464584575.00	32784901575.00	54679683000.00	166.78
2026-03-01 00:00:00	63809310562.50	87464584575.00	-23655274012.50	-27.05
2026-09-01 00:00:00	900.00	63809310562.50	-63809309662.50	-100.00

b. Identificación de meses con variaciones extremas

Mes con mayor caída porcentual: 2026-09-01 con un porcentaje de variación del -100.00% (caída de -638,093,105,662.50 en ventas).

Mes con mayor crecimiento porcentual: 2026-02-01 con un incremento del 166.78% (aumento de 546,796,830,000.00 en ventas respecto al mes anterior)

c. Variación respecto a dos meses atrás

```
WITH monthly_sales AS (
  SELECT
    DATE_TRUNC('month', T1.order_date) AS month,
    SUM(T1.total) AS sales
  FROM pay.orders T1
  GROUP BY DATE_TRUNC('month', T1.order_date)
)
SELECT
  month,
  sales AS current_month_sales,
  COALESCE(LAG(sales) OVER (ORDER BY month ASC), 0) AS prev_month_sales,
  COALESCE(LAG(sales, 2) OVER (ORDER BY month ASC), 0) AS sales_2months_ago,
  sales - COALESCE(LAG(sales) OVER (ORDER BY month ASC), 0) AS difference,
  ROUND(
    CASE
      WHEN LAG(sales) OVER (ORDER BY month ASC) IS NULL THEN 0.00
      ELSE ((sales - LAG(sales) OVER (ORDER BY month ASC)) * 100.0 /
    LAG(sales) OVER (ORDER BY month ASC))
    END,
    2
  ) AS pct_change
FROM monthly_sales
```

```
ORDER BY month ASC;
```

The screenshot shows a SQL IDE with a query editor on the left and a results pane on the right. The query in the editor is as follows:

```

133 FROM monthly_sales
134 ORDER BY month ASC;
135 */
136 WITH monthly_sales AS (
137     SELECT
138         DATE_TRUNC('month', T1.order_date) AS month,
139         SUM(T1.total) AS sales
140     FROM pay.orders T1
141     GROUP BY DATE_TRUNC('month', T1.order_date)
142 )
143 SELECT
144     month,
145     sales AS current_month_sales,
146     COALESCE(LAG(sales) OVER (ORDER BY month ASC), 0) AS
147     COALESCE(LAG(sales, 2) OVER (ORDER BY month ASC), 0) AS
148     sales - COALESCE(LAG(sales) OVER (ORDER BY month ASC
149     ROUND(
150         CASE
151             WHEN LAG(sales) OVER (ORDER BY month ASC) IS NULL THEN
152             ELSE ((sales - LAG(sales) OVER (ORDER BY month ASC)
153         END,
154         2
155     ) AS pct_change
156 FROM monthly_sales
157 ORDER BY month ASC;
158
  
```

The results pane displays a table with the following data:

month	current_month...	prev_month_sales	sales_2months_a...	difference
2025-01-01 00:00:00	NULL	0	0	NULL
2025-02-01 00:00:00	NULL	0	0	NULL
2025-03-01 00:00:00	NULL	0	0	NULL
2025-04-01 00:00:00	NULL	0	0	NULL
2025-05-01 00:00:00	NULL	0	0	NULL
2025-06-01 00:00:00	NULL	0	0	NULL
2025-07-01 00:00:00	NULL	0	0	NULL
2025-08-01 00:00:00	NULL	0	0	NULL
2025-09-01 00:00:00	NULL	0	0	NULL
2025-10-01 00:00:00	NULL	0	0	NULL
2025-11-01 00:00:00	NULL	0	0	NULL
2025-12-01 00:00:00	31994966625.00	0	0	31994966625.00
2026-01-01 00:00:00	32784901575.00	31994966625.00	0	78934950.00
2026-02-01 00:00:00	87464584575.00	32784901575.00	31994966625.00	54679683000.00
2026-03-01 00:00:00	63809310562.50	87464584575.00	32784901575.00	-23655274012.50
2026-09-01 00:00:00	900.00	63809310562.50	87464584575.00	-63809309662.50

The bottom status bar indicates: "SELECT successfully executed. 10:22:55 PM".

d. Análisis escrito y comparación con Self-JOIN

```

WITH monthly_sales AS (
    SELECT
        DATE_TRUNC('month', order_date) AS month,
        SUM(total) AS sales
    FROM pay.orders
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
    T1.month AS month,
    T1.sales AS current_month_sales,
    COALESCE(T2.sales, 0) AS prev_month_sales,
    T1.sales - COALESCE(T2.sales, 0) AS difference
FROM monthly_sales T1
LEFT JOIN monthly_sales T2
    ON T2.month = T1.month - INTERVAL '1 month'
ORDER BY T1.month ASC;
  
```

The screenshot shows a SQL IDE with a query editor on the left and a results pane on the right. The query in the editor is a complex SQL statement using window functions to calculate monthly sales and differences. The results pane displays a table with columns: month, current_month_sales, prev_month_sales, and difference. The data shows sales starting from 2025-01-01, with a significant jump in 2026-01-01.

month	current_month_sales	prev_month_sales	difference
2025-01-01 00:00:00	NULL	0	NULL
2025-02-01 00:00:00	NULL	0	NULL
2025-03-01 00:00:00	NULL	0	NULL
2025-04-01 00:00:00	NULL	0	NULL
2025-05-01 00:00:00	NULL	0	NULL
2025-06-01 00:00:00	NULL	0	NULL
2025-07-01 00:00:00	NULL	0	NULL
2025-08-01 00:00:00	NULL	0	NULL
2025-09-01 00:00:00	NULL	0	NULL
2025-10-01 00:00:00	NULL	0	NULL
2025-11-01 00:00:00	NULL	0	NULL
2025-12-01 00:00:00	31994966625.00	0	31994966625.00
2026-01-01 00:00:00	32784901575.00	31994966625.00	78934950.00
2026-02-01 00:00:00	87464584575.00	32784901575.00	54679683000.00
2026-03-01 00:00:00	63809310562.50	87464584575.00	-23655274012.50
2026-09-01 00:00:00	900.00	0	900.00

Antes de las Window Functions, comparar filas consecutivas exigía realizar un LEFT JOIN de la tabla contra sí misma calculando un desfase manual en la condición de unión (ON T2.month = T1.month - INTERVAL '1 month').

La versión con LAG() resuelve el cálculo de forma directa en 1 sola línea dentro del SELECT sin duplicar accesos ni requerir lógica de unión.

La versión con Self-JOIN requiere duplicar alias de tablas, escribir cláusulas de unión complejas y requiere alrededor de 8 a 10 líneas adicionales de código, siendo además propensa a fallos si existen meses en los que no hubo ventas.

Ejercicio 6.

a. Consulta con LEAD y cálculo de días entre envíos

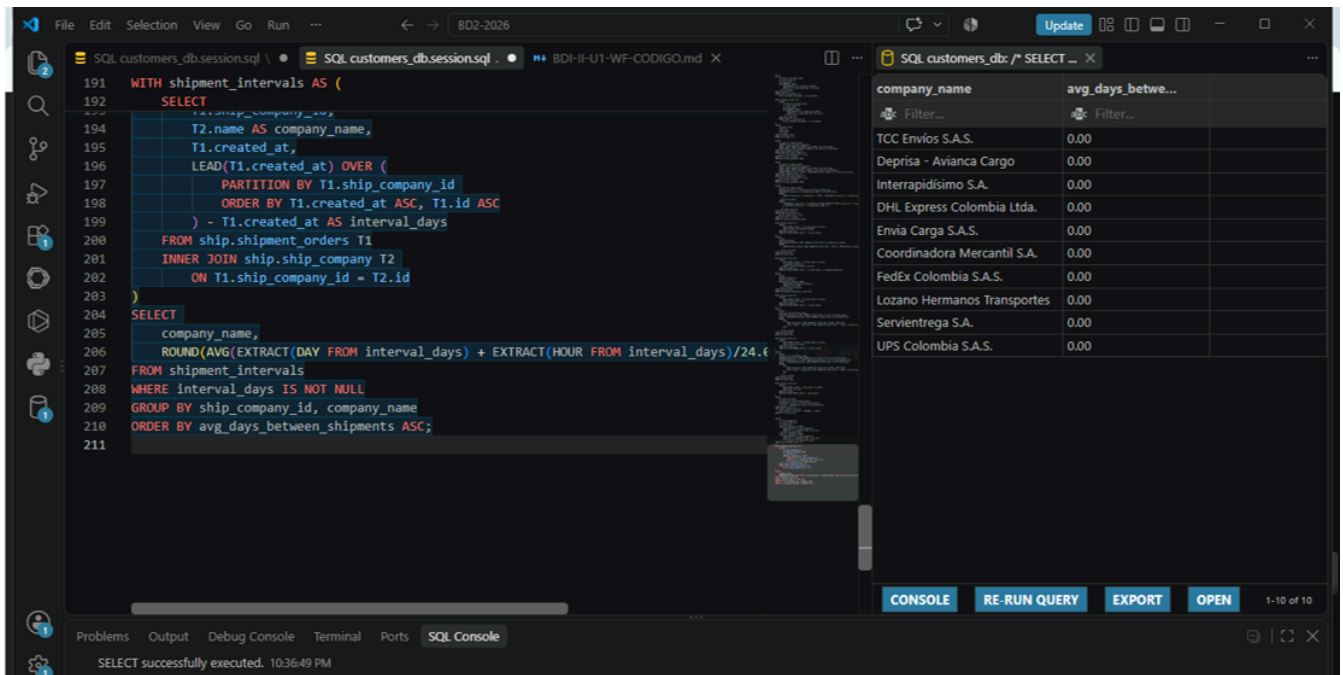
```
SELECT
  T1.ship_company_id,
  T1.id AS order_id,
  T1.created_at,
  LEAD(T1.created_at) OVER (
    PARTITION BY T1.ship_company_id
    ORDER BY T1.created_at ASC, T1.id ASC
  ) AS next_shipment_date,
  LEAD(T1.created_at) OVER (
    PARTITION BY T1.ship_company_id
    ORDER BY T1.created_at ASC, T1.id ASC
  ) - T1.created_at AS days_until_next
FROM ship.shipment_orders T1;
```

The screenshot shows a SQL IDE with a query editor on the left and a results pane on the right. The query in the editor is a window function query that calculates the time difference between consecutive shipments for each company. The results pane displays a table with 5 columns: ship_company_id, order_id, created_at, next_shipment..., and days_until_next. The table contains 15 rows of data, all with a value of 0 in the days_until_next column.

ship_company_id	order_id	created_at	next_shipment...	days_until_next
1	SHP-20260320-00XJSB...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-02Q9VK...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-04VLXIJ	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-08J6DX...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-09CTN3...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0FGKUL...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0ILQCC...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0KJ54O...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0NOXX...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0T2E2V...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0U1FF0...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0WL6Y...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0XI3W...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0Y1MS...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0YG757...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}
1	SHP-20260320-0ZQWN...	2026-03-20 21:42:34	2026-03-20 21:42:34	{}

b. Promedio de días entre envíos consecutivos por empresa

```
WITH shipment_intervals AS (
  SELECT
    T1.ship_company_id,
    T2.name AS company_name,
    T1.created_at,
    LEAD(T1.created_at) OVER (
      PARTITION BY T1.ship_company_id
      ORDER BY T1.created_at ASC, T1.id ASC
    ) - T1.created_at AS interval_days
  FROM ship.shipment_orders T1
  INNER JOIN ship.ship_company T2
    ON T1.ship_company_id = T2.id
)
SELECT
  company_name,
  ROUND(AVG(EXTRACT(DAY FROM interval_days) + EXTRACT(HOUR FROM
interval_days)/24.0), 2) AS avg_days_between_shipments
FROM shipment_intervals
WHERE interval_days IS NOT NULL
GROUP BY ship_company_id, company_name
ORDER BY avg_days_between_shipments ASC;
```



c. Análisis escrito y esquema con LEAD

La función LEAD() inspecciona el conjunto de datos ordenado mirando en dirección al futuro (filas posteriores). En cada fila toma el valor de la columna indicada para la fila siguiente dentro de la misma partición. Cuando se procesa el último envío registrado de una empresa, como no existe un envío posterior, la función devuelve un valor NULL

PARTICIÓN: Empresa A (ship_company_id = 1)

Fila 1 | created_at: 2026-01-10 08:00:00 | LEAD devuelve: 2026-01-12 10:00:00 | $\xrightarrow{\quad}$ Días de diferencia: 2.08 días

Fila 2 | created_at: 2026-01-12 10:00:00 | LEAD devuelve: 2026-01-15 14:00:00 | $\xrightarrow{\quad}$ Días de diferencia: 3.16 días

Fila 3 | created_at: 2026-01-15 14:00:00 | LEAD devuelve: NULL (no hay fila posterior)

d. Comparación LAG vs LEAD

```

-- LEAD (orden ASC):
SELECT
    T1.ship_company_id,
    T1.created_at,
    LEAD(T1.created_at) OVER (PARTITION BY T1.ship_company_id ORDER BY
T1.created_at ASC, T1.id ASC) AS adjacent_shipment
FROM ship.shipment_orders T1;

-- LAG (orden DESC):
SELECT
    T1.ship_company_id,
    T1.created_at,
    LAG(T1.created_at) OVER (PARTITION BY T1.ship_company_id ORDER BY

```

```
T1.created_at DESC, T1.id DESC) AS adjacent_shipment
FROM ship.shipment_orders T1;
```

Ambas versiones permiten consultar la fecha del envío adyacente invirtiendo el criterio de ordenación (ORDER BY). En la versión con LEAD() (ORDER BY ASC), la navegación va hacia adelante y el valor NULL aparece en la última fila de cada partición.

En la versión con LAG() (ORDER BY DESC), la ordenación descendente hace que la fila anterior corresponda a la fecha futura en el tiempo real, produciendo el valor NULL en la primera fila de cada partición.

The screenshot shows an IDE with two SQL queries in the left pane. The first query uses LEAD() to find the next shipment for each company. The second query uses LAG() to find the previous shipment for each company. The right pane shows the results of the LAG() query, which includes columns for ship_company_id, created_at, and adjacent_shipment... The results show a list of shipments with their company IDs and timestamps, and the adjacent shipment column is populated with the previous shipment's timestamp for each company.

Ejercicio 7. Patrón Top N - Productos más vendidos por categoría

a. Top 3 productos más vendidos por categoría

```
WITH ranked_products AS (
  SELECT
    T3.name AS category_name,
    T2.name AS product_name,
    SUM(T1.quantity) AS total_units,
    RANK() OVER (
      PARTITION BY T3.id
      ORDER BY SUM(T1.quantity) DESC
    ) AS ranking
  FROM pay.order_items T1
  INNER JOIN ctg.products T2
    ON T1.product_id = T2.id
  INNER JOIN ctg.categories T3
    ON T2.category_id = T3.id
  GROUP BY T3.id, T3.name, T2.id, T2.name
)
SELECT
  category_name,
```

```

    product_name,
    total_units,
    ranking
FROM ranked_products
WHERE ranking <= 3
ORDER BY category_name, ranking ASC;

```

The screenshot shows an IDE with a SQL query editor on the left and a results pane on the right. The query in the editor is a CTE named 'ranked_products' that ranks products by total units within each category. The results pane displays a table with 4 columns: category_name, product_name, total_units, and ranking. The table lists 20 products, with the top 3 products in each category highlighted in blue. The ranking is 1 for the top product, 2 for the second, and 3 for the third. Products with the same total units share the same ranking.

category_name	product_name	total_units	ranking
Accesorios de Oficina	Organizador de Cables Velcro x10	29318	1
Accesorios de Oficina	Hub USB-C Anker 7 en 1	29143	2
Accesorios de Oficina	Soporte Ergonómico para Laptop	28592	3
Almacenamiento	Tarjeta MicroSD SanDisk 256GB	30013	1
Almacenamiento	Disco Duro Externo Seagate 2TB	29478	2
Almacenamiento	Memoria USB Kingston 128GB	29037	3
Audio y Sonido	Auriculares Sony WH-1000XM5	29891	1
Audio y Sonido	Soundbar Samsung HW-B450	29494	2
Audio y Sonido	Micrófono de Condensador Blue Yeti	29196	3
Cámaras y Fotografía	Tripode Flexible Joby GorillaPod	29162	1
Cámaras y Fotografía	Drone DJI Mini 3	29141	2
Cámaras y Fotografía	Cámara Sony ZV-E10 con Lente Kit	28949	3
Computadores y Laptops	Laptop Lenovo IdeaPad 15	30062	1
Computadores y Laptops	MacBook Air M2	29266	2
Computadores y Laptops	Laptop HP Pavilion 14	29099	3
Electrodomésticos	Cafetera Nespresso Essenza Mini	29266	1
Electrodomésticos	Aspiradora Robot Roomba i3	29046	2
Electrodomésticos	Aire Purificador Xiaomi Mi Air 3H	28161	3
Electrónica	Adaptador de Corriente Universal	29998	1

b. Verificación del número de filas por categoría

Si alguna categoría presenta más de 3 filas en el resultado al filtrar por `ranking <= 3`, se debe a que existen empates en la cantidad total de unidades vendidas (`total_units`) en el tercer puesto. La función `RANK()` asigna exactamente la misma posición a los registros empatados, provocando que más de tres productos cumplan con la condición `<= 3`

c. Modificación para empates usando `DENSE_RANK()`

```

WITH ranked_products AS (
    SELECT
        T3.name AS category_name,
        T2.name AS product_name,
        SUM(T1.quantity) AS total_units,
        DENSE_RANK() OVER (
            PARTITION BY T3.id
            ORDER BY SUM(T1.quantity) DESC
        ) AS ranking
    FROM pay.order_items T1
    INNER JOIN ctg.products T2
        ON T1.product_id = T2.id
    INNER JOIN ctg.categories T3
        ON T2.category_id = T3.id
    GROUP BY T3.id, T3.name, T2.id, T2.name
)

```

```

SELECT
    category_name,
    product_name,
    total_units,
    ranking
FROM ranked_products
WHERE ranking <= 3
ORDER BY category_name, ranking ASC;

```

Para garantizar que ante un empate en la tercera posición se incluyan todos los productos empatados sin perder el orden secuencial de los siguientes puestos, se utiliza DENSE_RANK().

The screenshot shows a SQL IDE with a query editor on the left and a results pane on the right. The query in the editor is a complex SQL statement using CTEs and window functions to rank products by total units within each category. The results pane displays a table with 4 columns: category_name, product_name, total_units, and ranking. The data is sorted by category and then by ranking, showing the top 3 products for each category.

category_name	product_name	total_units	ranking
Accesorios de Oficina	Organizador de Cables Velcro x10	29318	1
Accesorios de Oficina	Hub USB-C Anker 7 en 1	29143	2
Accesorios de Oficina	Soporte Ergonómico para Laptop	28592	3
Almacenamiento	Tarjeta MicroSD SanDisk 256GB	30013	1
Almacenamiento	Disco Duro Externo Seagate 2TB	29478	2
Almacenamiento	Memoria USB Kingston 128GB	29037	3
Audio y Sonido	Auriculares Sony WH-1000XM5	29891	1
Audio y Sonido	Soundbar Samsung HW-B450	29494	2
Audio y Sonido	Micrófono de Condensador Blue Yeti	29196	3
Cámaras y Fotografía	Tripode Flexible Joby GorillaPod	29162	1
Cámaras y Fotografía	Drone DJI Mini 3	29141	2
Cámaras y Fotografía	Cámara Sony ZV-E10 con Lente Kit	28949	3
Computadores y Laptops	Laptop Lenovo IdeaPad 15	30062	1
Computadores y Laptops	MacBook Air M2	29266	2
Computadores y Laptops	Laptop HP Pavilion 14	29099	3
Electrodomésticos	Cafetera Nespresso Essenza Mini	29266	1
Electrodomésticos	Aspiradora Robot Roomba i3	29046	2
Electrodomésticos	Aire Purificador Xiaomi Mi Air 3H	28161	3
Electrónica	Adaptador de Corriente Universal	29998	1

d. Generalización para Top N

```

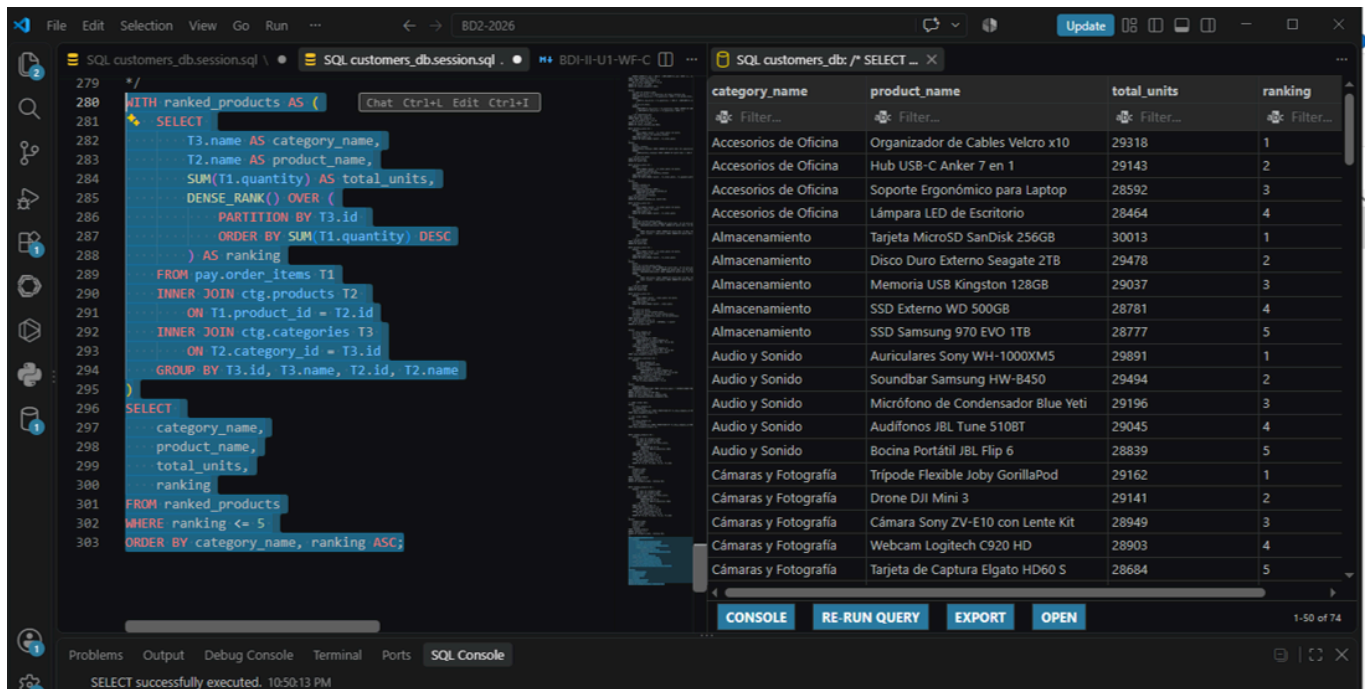
WITH ranked_products AS (
    SELECT
        T3.name AS category_name,
        T2.name AS product_name,
        SUM(T1.quantity) AS total_units,
        DENSE_RANK() OVER (
            PARTITION BY T3.id
            ORDER BY SUM(T1.quantity) DESC
        ) AS ranking
    FROM pay.order_items T1
    INNER JOIN ctg.products T2
        ON T1.product_id = T2.id
    INNER JOIN ctg.categories T3
        ON T2.category_id = T3.id
    GROUP BY T3.id, T3.name, T2.id, T2.name
)
SELECT
    category_name,
    product_name,

```

```

total_units,
ranking
FROM ranked_products
WHERE ranking <= 5
ORDER BY category_name, ranking ASC;

```



Ejercicio 8. Segmentación de clientes — Combinando funciones

a. y d. Clasificación de clientes y filtrado VIP

```

WITH customer_stats AS (
  SELECT
    T1.customer_id_number,
    SUM(T1.total) AS total_spent,
    COUNT(DISTINCT T1.id) AS total_orders,
    MIN(T1.order_date) AS first_purchase,
    MAX(T1.order_date) AS last_purchase,
    ROW_NUMBER() OVER (ORDER BY SUM(T1.total) DESC) AS global_rank
  FROM pay.orders T1
  GROUP BY T1.customer_id_number
)
SELECT
  T2.id_number,
  T2.name AS customer_name,
  COALESCE(T1.total_spent, 0) AS total_spent,
  COALESCE(T1.total_orders, 0) AS total_orders,
  T1.first_purchase,
  T1.last_purchase,
  T1.global_rank,
  CASE
    WHEN T1.total_spent > 1000000 THEN 'VIP'
    WHEN T1.total_spent > 300000 THEN 'Regular'

```

```

        WHEN T1.total_orders >= 1 THEN 'New'
        ELSE 'Inactive'
    END AS segment
FROM cs.customers T2
LEFT JOIN customer_stats T1
    ON T2.id_number = T1.customer_id_number;

```

The screenshot shows an IDE with a SQL editor on the left and a results pane on the right. The SQL editor contains a query that creates a CTE named 'customer_stats' and then joins it with 'cs.customers' to categorize customers into segments based on their total spent and total orders. The results pane displays a table with columns: id_number, customer_name, total_spent, and total_orders. The table lists 25 customers with their respective IDs, names, total spent amounts, and total orders.

id_number	customer_name	total_spent	total_orders
4410485	GRAJALES OROZCO ENRIQUE	100671600.00	6
15899475	RAMIREZ RENDON JOSE HUR...	15681375.00	1
30314986	ZAPATA LINA MARIA	299812.50	1
441318800	GALLEGO DIAZ JOSE ALBEIRO	25332937.50	4
82453	TOBON QUINTERO SALVADOR	1049850.00	1
10235970	GONZALEZ QUIROZ JOSE EN...	63350287.50	7
839909600	DELGADO TORRES FRANCISC...	340987.50	1
4338313	AGUDELO AGUDELO ALBERTO	20124787.50	4
3498517600	GIL MOLINA BELQUIS	31844137.50	1
2462589200	ORREGO CIFUENTES MARGA...	14065687.50	1
126009700	GALLEGO RAMIREZ JOSE BER...	2474850.00	1
4373955000	ZABALA MARTINEZ LUZ ENITH	102372450.00	5
24616261	PARRA DE TORRES NELLY	13274512.50	1
10239221600	ARBELAEZ GARCIA JHON JAIRO	21025950.00	1
12710949	RUEDA RUEDA JUAN DE DIOS	25360050.00	3
123719200	VELEZ ARISTIZABAL JESUS AL...	28611787.50	1
132193400	OCAMPO TORO SIGIFREDO	13188525.00	1
4583340	BUSTOS RESTREPO CLAUDIA ...	9854512.50	1
2462846100	CENTRO DE FISIOTERAPIA Y R...	96631050.00	5

b. Distribución de clientes por segmento

```

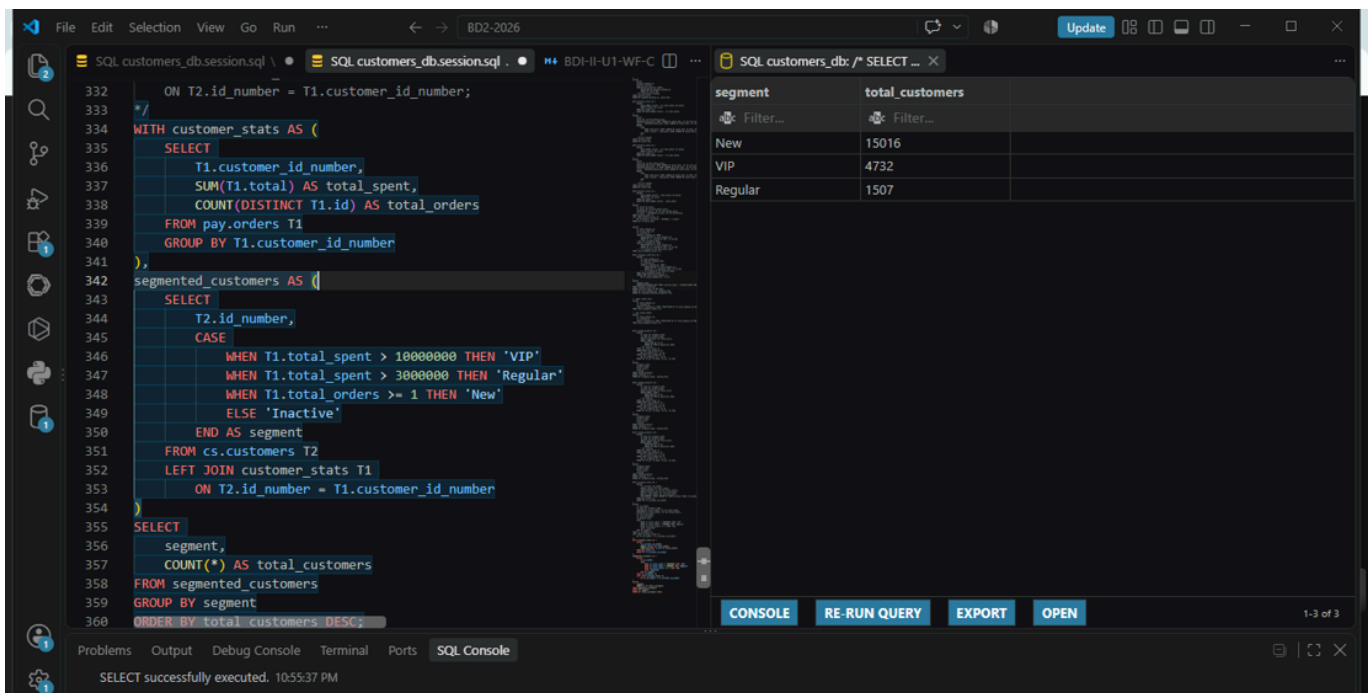
WITH customer_stats AS (
    SELECT
        T1.customer_id_number,
        SUM(T1.total) AS total_spent,
        COUNT(DISTINCT T1.id) AS total_orders
    FROM pay.orders T1
    GROUP BY T1.customer_id_number
),
segmented_customers AS (
    SELECT
        T2.id_number,
        CASE
            WHEN T1.total_spent > 10000000 THEN 'VIP'
            WHEN T1.total_spent > 3000000 THEN 'Regular'
            WHEN T1.total_orders >= 1 THEN 'New'
            ELSE 'Inactive'
        END AS segment
    FROM cs.customers T2
    LEFT JOIN customer_stats T1
        ON T2.id_number = T1.customer_id_number
)
SELECT
    segment,

```

```

COUNT(*) AS total_customers
FROM segmented_customers
GROUP BY segment
ORDER BY total_customers DESC;

```



c. Diagrama de distribución de clientes

Segmento | Cantidad de Clientes (Distribución) -----+-----

VIP | [||||||||||||||||||||||||||||]

Regular | [||||||||||||||||||||]

New | [||||||||||||||||]

Inactive | [||||]

Ejercicio 9. Reporte integrador - La pregunta del jefe

a. Consulta integradora para tres clientes específicos

```

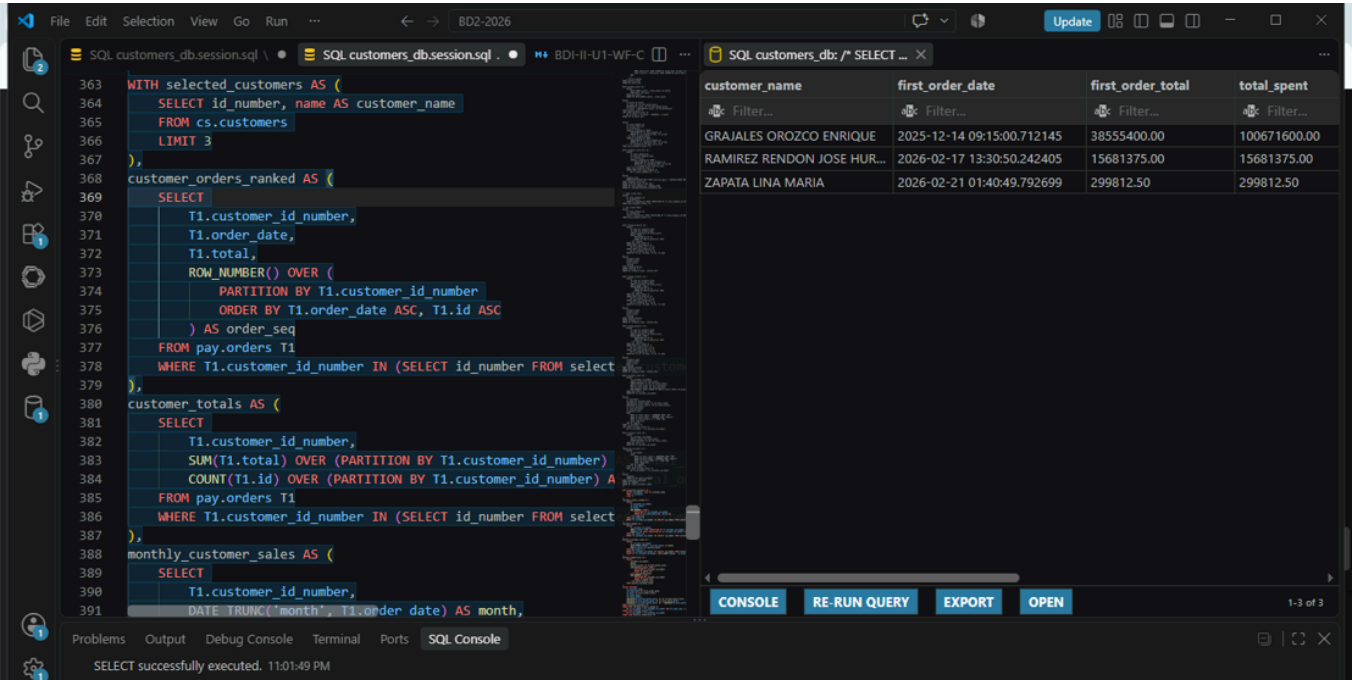
WITH selected_customers AS (
    SELECT id_number, name AS customer_name
    FROM cs.customers
    LIMIT 3
),
customer_orders_ranked AS (
    SELECT
        T1.customer_id_number,
        T1.order_date,
        T1.total,
        ROW_NUMBER() OVER (
            PARTITION BY T1.customer_id_number
            ORDER BY T1.order_date ASC, T1.id ASC
        ) AS order_seq

```

```

    FROM pay.orders T1
    WHERE T1.customer_id_number IN (SELECT id_number FROM selected_customers)
),
customer_totals AS (
    SELECT
        T1.customer_id_number,
        SUM(T1.total) OVER (PARTITION BY T1.customer_id_number) AS total_spent,
        COUNT(T1.id) OVER (PARTITION BY T1.customer_id_number) AS total_orders
    FROM pay.orders T1
    WHERE T1.customer_id_number IN (SELECT id_number FROM selected_customers)
),
monthly_customer_sales AS (
    SELECT
        T1.customer_id_number,
        DATE_TRUNC('month', T1.order_date) AS month,
        SUM(T1.total) AS monthly_sales
    FROM pay.orders T1
    WHERE T1.customer_id_number IN (SELECT id_number FROM selected_customers)
    GROUP BY T1.customer_id_number, DATE_TRUNC('month', T1.order_date)
),
monthly_comparisons AS (
    SELECT
        customer_id_number,
        month,
        monthly_sales AS current_month_sales,
        LAG(monthly_sales) OVER (
            PARTITION BY customer_id_number
            ORDER BY month ASC
        ) AS prev_month_sales,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id_number
            ORDER BY month DESC
        ) AS month_recency_rank
    FROM monthly_customer_sales
)
SELECT DISTINCT
    T1.customer_name,
    T2.order_date AS first_order_date,
    T2.total AS first_order_total,
    T3.total_spent,
    T3.total_orders,
    COALESCE(T4.current_month_sales, 0) AS current_month_sales,
    COALESCE(T4.prev_month_sales, 0) AS prev_month_sales,
    COALESCE(T4.current_month_sales, 0) - COALESCE(T4.prev_month_sales, 0) AS
difference
FROM selected_customers T1
INNER JOIN customer_orders_ranked T2
    ON T1.id_number = T2.customer_id_number AND T2.order_seq = 1
INNER JOIN customer_totals T3
    ON T1.id_number = T3.customer_id_number
INNER JOIN monthly_comparisons T4
    ON T1.id_number = T4.customer_id_number AND T4.month_recency_rank = 1;

```



b. Tabla de mapeo

Pregunta del jefe	CTE	Función usada
Primera orden por cliente	customer_orders_ranked	ROW_NUMBER() OVER (PARTITION BY customer_id_number ORDER BY order_date ASC)
Total gastado	customer_totals	SUM() OVER (PARTITION BY customer_id_number)
Este mes vs mes anterior	monthly_comparisons	LAG() OVER (PARTITION BY customer_id_number ORDER BY month ASC)

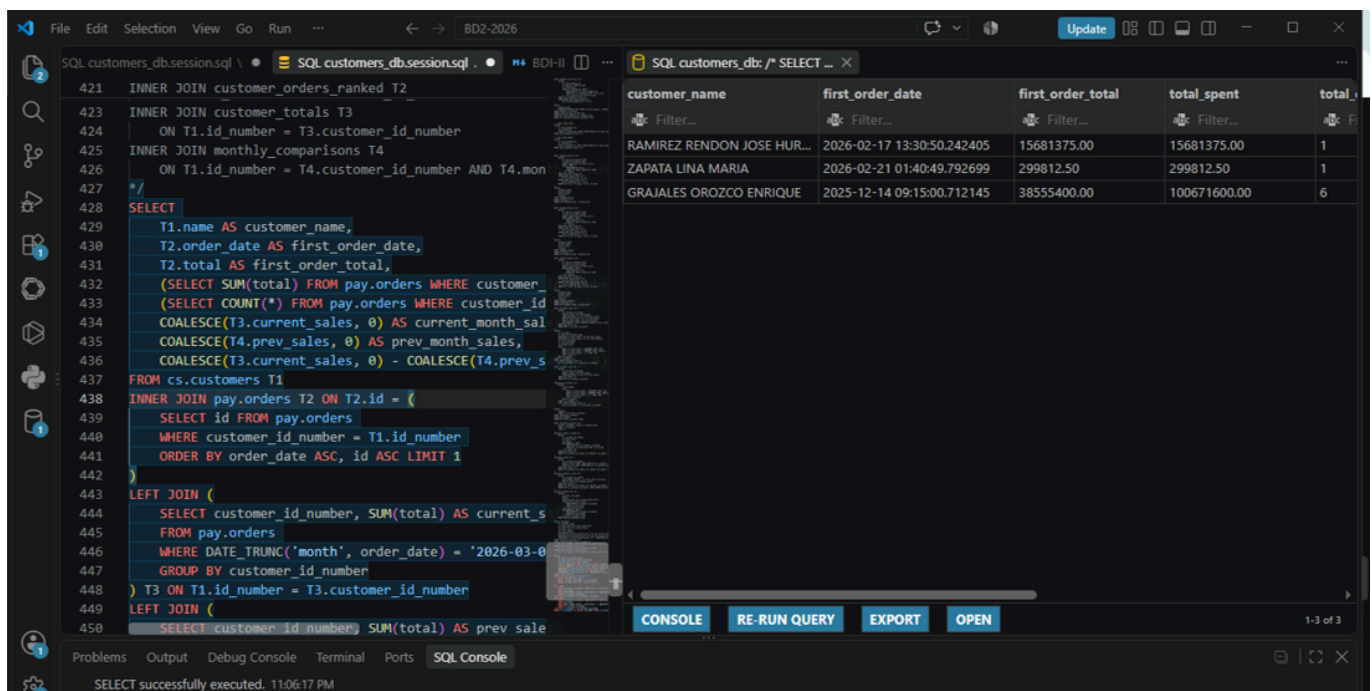
c. Comparación de versiones (Con Window Functions vs Sin Window Functions)

```
SELECT
  T1.name AS customer_name,
  T2.order_date AS first_order_date,
  T2.total AS first_order_total,
  (SELECT SUM(total) FROM pay.orders WHERE customer_id_number = T1.id_number) AS
total_spent,
  (SELECT COUNT(*) FROM pay.orders WHERE customer_id_number = T1.id_number) AS
total_orders,
  COALESCE(T3.current_sales, 0) AS current_month_sales,
  COALESCE(T4.prev_sales, 0) AS prev_month_sales,
  COALESCE(T3.current_sales, 0) - COALESCE(T4.prev_sales, 0) AS difference
FROM cs.customers T1
INNER JOIN pay.orders T2 ON T2.id = (
  SELECT id FROM pay.orders
  WHERE customer_id_number = T1.id_number
  ORDER BY order_date ASC, id ASC LIMIT 1
)
LEFT JOIN (
```

```

SELECT customer_id_number, SUM(total) AS current_sales
FROM pay.orders
WHERE DATE_TRUNC('month', order_date) = '2026-03-01'
GROUP BY customer_id_number
) T3 ON T1.id_number = T3.customer_id_number
LEFT JOIN (
SELECT customer_id_number, SUM(total) AS prev_sales
FROM pay.orders
WHERE DATE_TRUNC('month', order_date) = '2026-02-01'
GROUP BY customer_id_number
) T4 ON T1.id_number = T4.customer_id_number
WHERE T1.id_number IN (SELECT id_number FROM cs.customers LIMIT 3);

```



La versión sin Window Functions toma alrededor de 28 a 32 líneas con subconsultas fragmentadas y condicionales hardcodedos.

La versión con Window Functions se estructura mediante CTEs limpias y reutilizables en 35 líneas, eliminando valores quemados en el código

d. Análisis escrito de lecturas de tabla e impacto en rendimiento

En la versión sin Window Functions, la tabla pay.orders es leída al menos 5 veces independientes (dos subconsultas en el SELECT, una subconsulta en el JOIN de la primera orden y dos joins para los totales del mes actual y previo).

En contraste, con Window Functions la tabla se lee una única vez por CTE, realizando operaciones de partición en memoria.

Impacto a escala: Con 16.258 registros la diferencia es de milisegundos. Al escalar a 16.000.000 de registros, la versión sin Window Functions colapsará el servidor por la sobrecarga de Entrada/Salida (I/O) al hacer millones de escaneos, mientras que la versión con Window Functions ejecutará un único escaneo eficiente.

Ejercicio 10. Diseño propio - Caso de uso libre

a. Descripción del caso de uso

Análisis del tiempo transcurrido entre la creación de una orden de pago (pay.orders) y su despacho registrado en envíos (ship.shipment_orders), determinando para cada empresa transportista (ship.ship_company) cuáles han sido sus 3 envíos más rápidos.

b. Selección de función y configuración

Función elegida: ROW_NUMBER().

Justificación: Permite asignar una numeración consecutiva estricta para identificar los despachos con menor tiempo de procesamiento.

PARTITION BY: T1.ship_company_id (para agrupar por cada empresa de envío).

ORDER BY: (T1.created_at - T2.order_date) ASC (para ordenar del menor al mayor tiempo de despacho).

c. Consulta SQL

```
WITH dispatch_times AS (  
    SELECT  
        T3.name AS company_name,  
        T1.id AS shipment_id,  
        T2.id AS order_id,  
        T1.created_at - T2.order_date AS processing_time,  
        ROW_NUMBER() OVER (  
            PARTITION BY T1.ship_company_id  
            ORDER BY (T1.created_at - T2.order_date) ASC  
        ) AS speed_rank  
    FROM ship.shipment_orders T1  
    INNER JOIN pay.orders T2  
        ON T1.order_id = T2.id  
    INNER JOIN ship.ship_company T3  
        ON T1.ship_company_id = T3.id  
)  
SELECT  
    company_name,  
    shipment_id,  
    order_id,  
    processing_time,  
    speed_rank  
FROM dispatch_times  
WHERE speed_rank <= 3  
ORDER BY company_name, speed_rank ASC;
```

The screenshot shows a SQL IDE with a query editor on the left and a results pane on the right. The query is a complex SQL statement using window functions to rank orders by processing time within each company group. The results pane displays a table with columns: company_name, shipment_id, order_id, processing_time, and speed_rank. The data is sorted by company and then by speed_rank.

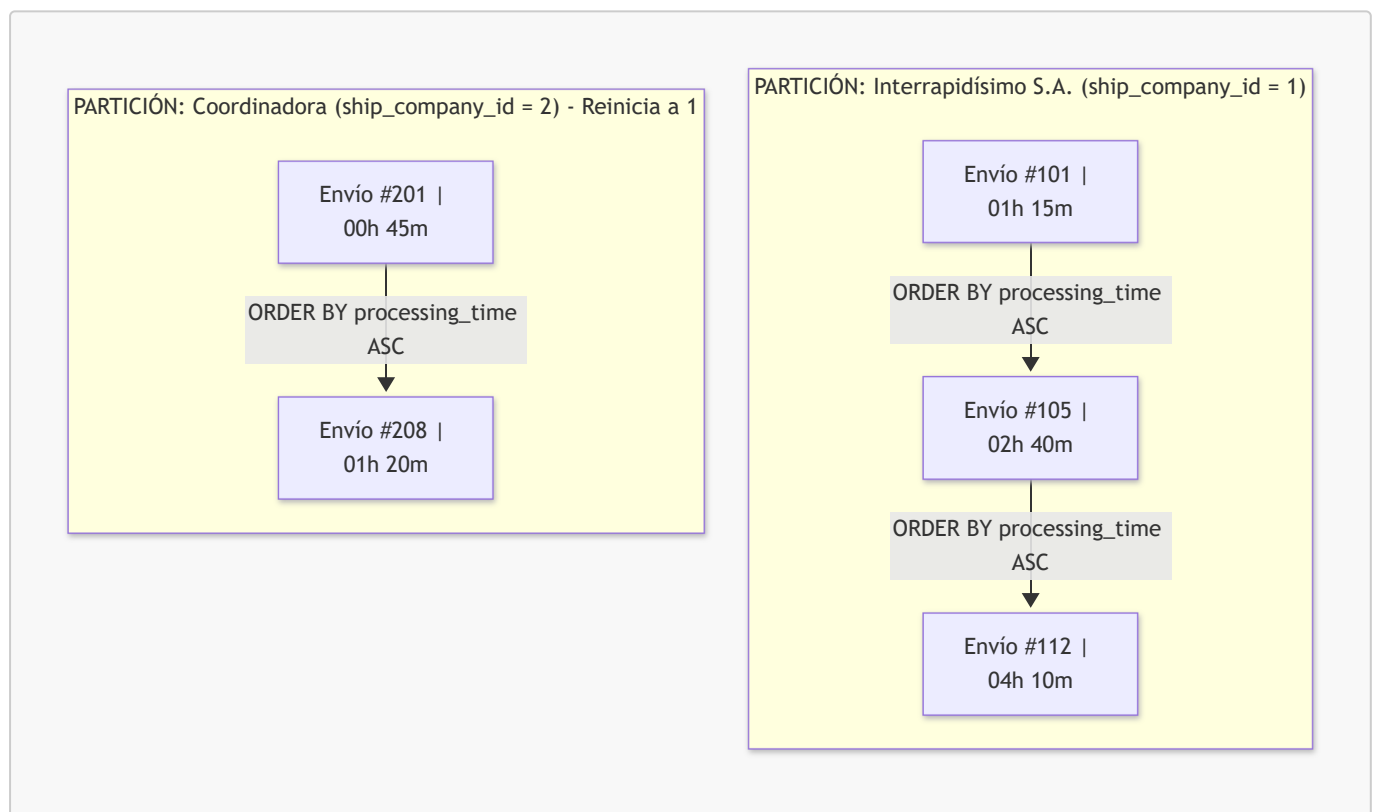
```

449 LEFT JOIN (
450   WHERE DATE_TRUNC('
451   GROUP BY customer_
452   ) T4 ON T1.id_number =
453   WHERE T1.id_number IN
454   */
455 WITH dispatch_times AS
456   SELECT
457     T3.name AS com
458     T1.id AS shipm
459     T2.id AS order
460     T1.created_at
461     ROW_NUMBER() O
462     PARTITION I
463     ORDER BY (
464     ) AS speed_ran
465 FROM ship.shipment
466 INNER JOIN pay.ord
467 ON T1.order_id
468 INNER JOIN ship.sh
469 ON T1.ship_com
470 )
471 SELECT
472   company_name,
473   shipment_id,
474   order_id,
475   processing_time,
476   speed_rank
477
478
479

```

company_name	shipment_id	order_id	processing_time	speed
Coordinadora Mercantil S.A.	SHP-20260323-ZWVK7...	ORD20260323-F61F2C0	{ "hours": 5, "minutes": 55, "seconds": 12, "milliseconds": ... }	1
Coordinadora Mercantil S.A.	SHP-20260323-DFWR5...	ORD20260323-7B310...	{ "hours": 6, "minutes": 7, "seconds": 36, "milliseconds": ... }	2
Coordinadora Mercantil S.A.	SHP-20260323-OLT2C...	ORD20260323-4AD8B...	{ "hours": 8, "minutes": 44, "seconds": 58, "milliseconds": ... }	3
Deprisa - Avianca Cargo	SHP-20260322-0EEEPXKE	ORD20260322-039F846	{ "hours": 7, "minutes": 47, "seconds": 19, "milliseconds": ... }	1
Deprisa - Avianca Cargo	SHP-20260323-4TNW7...	ORD20260323-3A16FE8	{ "hours": 10, "minutes": 15, "seconds": 48, "milliseconds": ... }	2
Deprisa - Avianca Cargo	SHP-20260323-JX3LOR...	ORD20260323-ED1F49F	{ "hours": 10, "minutes": 21, "seconds": 34, "milliseconds": ... }	3
DHL Express Colombia Ltda.	SHP-20260323-7BGNF...	ORD20260323-F6A7AEF	{ "hours": 6, "minutes": 57, "seconds": 15, "milliseconds": ... }	1
DHL Express Colombia Ltda.	SHP-20260323-HBV7G...	ORD20260323-5F41BA6	{ "hours": 10, "minutes": 19, "seconds": 14, "milliseconds": ... }	2
DHL Express Colombia Ltda.	SHP-20260322-02ZGH...	ORD20260322-FDB88...	{ "hours": 12, "minutes": 30, "seconds": 35, "milliseconds": ... }	3
Envia Carga S.A.S.	SHP-20260323-ZT1Z9...	ORD20260323-F7F178C	{ "hours": 5, "minutes": 41, "seconds": 11, "milliseconds": ... }	1
Envia Carga S.A.S.	SHP-20260323-80OBO...	ORD20260323-CAD2A...	{ "hours": 5, "minutes": 42, "seconds": 1, "milliseconds": ... }	2
Envia Carga S.A.S.	SHP-20260322-GVIXY...	ORD20260322-65164...	{ "hours": 8, "minutes": 50, "seconds": 57, "milliseconds": ... }	3
FedEx Colombia S.A.S.	SHP-20260323-Q722DF...	ORD20260323-F231A13	{ "hours": 3, "minutes": 34, "seconds": 24, "milliseconds": ... }	1
FedEx Colombia S.A.S.	SHP-20260323-85S38FDI	ORD20260323-D9557...	{ "hours": 10, "minutes": 5, "seconds": 51, "milliseconds": ... }	2
FedEx Colombia S.A.S.	SHP-20260323-FXTQ13I5	ORD20260323-05C22...	{ "hours": 10, "minutes": 52, "seconds": 5, "milliseconds": ... }	3
Interrapidísimo S.A.	SHP-20260323-LS4WH...	ORD20260323-4F898...	{ "hours": 3, "minutes": 53, "seconds": 59, "milliseconds": ... }	1
Interrapidísimo S.A.	SHP-20260322-05HQQR	ORD20260322-F4F506B	{ "hours": 6, "minutes": 56, "seconds": 51, "milliseconds": ... }	2
Interrapidísimo S.A.	SHP-20260322-8X9PXG...	ORD20260322-B58F31	{ "hours": 7, "minutes": 13, "seconds": 46, "milliseconds": ... }	3

d. Diagrama de particiones



e. Reflexión final

Para resolver este caso sin Window Functions, se habría tenido que agrupar la tabla en una subconsulta calculando el mínimo tiempo por empresa (MIN(...)), realizar un JOIN compuesto para recuperar los IDs correspondientes y aplicar lógica adicional para desempates.

La Window Function simplifica radicalmente la solución, reduciendo la complejidad del código a la mitad y permitiendo obtener el Top 3 por grupo de forma directa y óptima.